

BioDB仕様書

要求仕様・設計概要・運用方針

DolyLab

Table of contents

1 BioDB仕様書	4
1.1 本書の目的	4
1.2 背景	4
1.3 適用範囲(スコープ)	4
1.4 文書構成	5
1.5 仕様の要約	5
1.6 記法・約束事	5
2 システム全体構成	6
2.1 概要	6
2.2 システム構成図	7
2.3 構成要素一覧	7
2.4 ルーティング設計	8
3 認証・認可	9
3.1 位置づけと注意	9
3.2 設計動機(要旨)	9
3.3 トークン・ロールの概念(整理)	9
3.4 ルーティング(Auth 配下)	9
3.5 エンドポイント一覧(概要)	10
3.6 バリデーションと規約	10
4 センサ API	11
4.1 概要	11
4.2 エンドポイント一覧(概要)	11
4.3 データ形式・圧縮	11
4.3.1 送受信コンテナ(リクエスト/レスポンスの外枠)	11
4.3.2 素データ	12
4.3.3 formatとcompressionのアルゴリズム	12
4.4 リクエスト仕様	12
4.4.1 /sensor/data/write (POST)	12
4.4.2 /sensor/data/read (POST)	13
4.5 実装上の注意	13
5 イベント API	14
5.1 概要	14
5.2 エンドポイント一覧(概要)	14
5.3 リクエスト仕様・バリデーション	14
5.3.1 作成: /event/events(POST)	14
5.3.2 取得: /event/events (GET)	15
5.3.3 削除: /event/events/<event_id> (DELETE)	16
5.3.4 更新: event/events/<event_id> (POST)	16
5.4 実装上の注意	16

6	WebUI UI仕様	17
6.1	概要	17
6.2	各画面のUI仕様	17
6.2.1	ログイン画面	17
6.2.2	ユーザ情報ページ	18
6.2.3	被計測者管理画面	18
6.2.4	長期トークン管理画面	20
7	util UI仕様	23
7.1	概要	23
7.2	各画面のUI仕様	23
7.2.1	リアルタイムデータチェッカー	23
7.2.2	シンプルデータチェッカー	24
7.2.3	簡易イベント作成	25
7.2.4	簡易イベント編集ページ	26
7.2.5	イベントデータと時系列プロット	27
7.2.6	イベントデータと感情マップ	30
8	付録(OpenAPI仕様)	32
8.1	認証API	32
8.2	センサAPI	61
8.3	イベントAPI	68

1 BioDB仕様書

要求仕様・設計概要・運用方針

! Important

本書は，BioDBシステムの要求仕様，設計概要，運用方針をまとめたものである．
詳細なAPI定義は付録「OpenAPI仕様」(Chapter 8)に掲載する．

1.1 本書の目的

本書は，研究データの再現性・共有性・運用性を高めることを目的とした**BioDB システム**について，非機能要件を含む要求仕様および全体アーキテクチャを明示する．

1.2 背景

研究室内外で生成される生体センサデータは，計測条件・形式・保存場所が分散しやすく，再利用や横断分析の阻害要因となる．本システムは以下を満たすことで課題の解消を図る．

- センサデータの統一的な受け入れ口(API)を提供すること
- 管理UIおよび可視化クライアントにより，データの把握・確認を容易にすること
- 運用の単純化(静的配信 + API)により，学内サーバでの継続的な運用を可能にすること

1.3 適用範囲(スコープ)

- BioDBシステムのAPI群
- WebUI(/WebUI/)
- 可視化クライアント(/util/)
- データベース群

1.4 文書構成

- 構成概要(Chapter 2): システム全体図と要素の役割
- 機能要件(Chapter 3, Chapter 4, Chapter 5): 認証, センサ, イベント
- クライアント(Chapter 6, Chapter 7): 管理UIと可視化クライアント
- 付録(Chapter 8): OpenAPI 仕様(PDF貼付)

1.5 仕様の要約

- 配信: 静的配信(SSG) + CSR. /WebUI/ と /util/ は NGINX より配信
- API: FastAPI/Flaskによる /auth, /sensor, /event を提供
- データ: VictoriaMetrics / PostgreSQL / MongoDB に保存
- 認証: Google Identity Services + 独自JWT(短命Access)
- 可視化: ブラウザ上でAPIから取得し描画(CSR)

1.6 記法・約束事

- 本文の JSON は例示であり, 省略記号 ... を含むことがある
- API 名は /path/to/endpoint の形式で示す
- セキュリティ上の理由から一部値(鍵・トークン)は伏せ字, 単一文字列での表記とする

2 システム全体構成

2.1 概要

本システムは、APIサーバ群，静的配信によるWebユーザインターフェイス(WebUI)，およびデータベース群から構成される。
NGINXを中心としたリバースプロキシ構成であり，安全性と拡張性を重視した設計となっている。

2.2 システム構成図

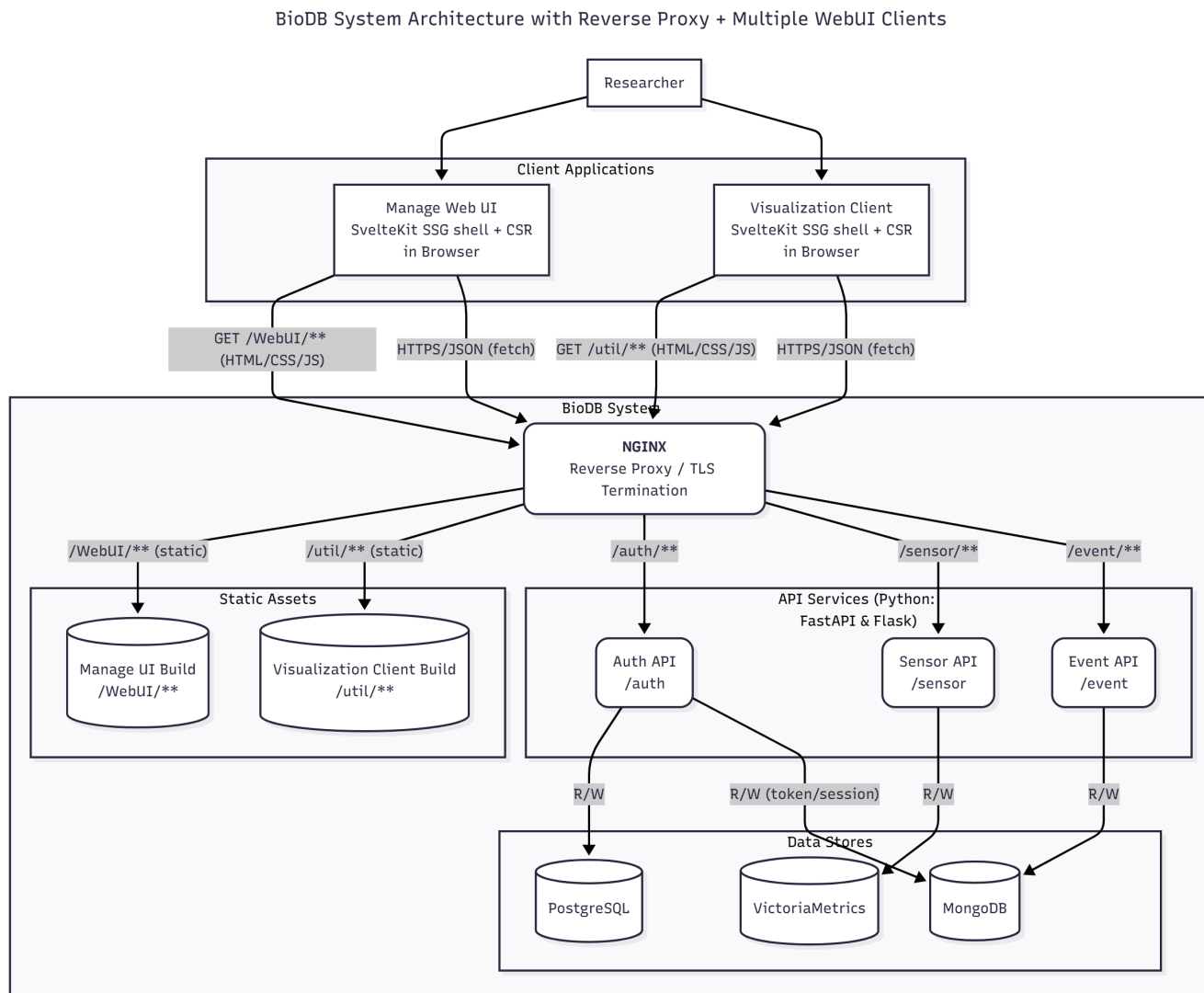


Figure 2.1: BioDBシステム構成図

2.3 構成要素一覧

要素	役割	主な責務	非責務
NGINX	逆プロキシ, TLS終端, 静的配信	/WebUI/** /util/** の静的配信, APIのパス振り分け	ビジネスロジック, テンプレート描画
WebUI(/WebUI/)	管理UI (SSGシェル+CSR)	認証UI, 設定操作, API連携	サーバサイドレンダリング, 長期ジョブ
可視化クライアント (/util/)	データ可視化 (SSGシェル+CSR)	センサ・イベントの取得, 重い集計(必要ならAPI側で実施) グラフ描画	

要素	役割	主な責務	非責務
Auth API	認証・認可	Google Identity検証, JWT発行/更新, 権限確認	UI, 静的配信
Sensor API	センサ入出力	バルク受信, 圧縮展開, 時系列書込/読出	権限管理(トークン検証のみ)
Event API	イベント管理	イベント登録/検索, メタ/トークン参照	認証(トークン発行はAuth API)
VictoriaMetrics	時系列データ保管	センサデータ書込/読出	アプリロジック
PostgreSQL	構造化データ保管	ユーザ, 被計測者データの保管	アプリロジック
MongoDB	半構造化データ保管	長期トークン, イベントデータの保管	アプリロジック

2.4 ルーティング設計

パス	種別	配信元/宛先
/WebUI/**	静的 (SPA)	NGINX → SSGビルド
/util/**	静的 (SPA)	NGINX → SSGビルド
/auth/**	API	NGINX → Auth API
/sensor/**	API	NGINX → Sensor API
/event/**	API	NGINX → Event API

3 認証・認可

3.1 位置づけと注意

Auth API は、JWT の発行／ユーザ・被計測者管理／長期トークン管理を担う。

Warning

注意：jwt 発行以外の API は主に WebUI から利用される。
一般クライアントからの直接利用は限定的である。

3.2 設計動機(要旨)

- 生体情報は個人情報であり、権限のない第三者アクセスを禁止する必要がある。
- 外出先・移動中の計測(スマートウォッチ等)を想定し、インターネット経由での安全な利用が前提となる。
- 将来的な分散運用(分散 DB / k8s 等)を見据え、認可は JWT によって疎結合化する。

本プロジェクトでは「認証は通常フロー+Google連携」「認可はJWT(用途別クレーム)」とし、サービスを機能分割してスケール・運用性を高める方針である。

3.3 トークン・ロールの概念(整理)

Auth API では用途別に 属性(WebUI / WebService / Token 認証) を使い分ける。

- WebUI：管理 UI からの操作／ユーザ・被計測者・長期トークンの管理
- WebService：外部Webアプリ等からのサービス向け発行
- Token認証：ユーザが作成した長期トークンに基づき、用途限定のJWT (sensor read/sensor write/event) を二次発行するフロー(後述)

3.4 ルーティング(Auth 配下)

NGINX にて /auth/ にパス振り分けされる(実装側の内部パスは /auth を省略)。本章では利用者視点で /auth 付きで記載する。

3.5 エンドポイント一覧(概要)

エンドポイント	説明	メソッド	必要な JWT
/auth/user/info	ユーザ情報の返却	GET	WebUI or WebService
/auth/user/info	ユーザ情報の更新	POST	WebUI
/auth/user	ユーザの作成	POST	WebUI (管理者)
/auth/participant	被計測者の作成	POST	WebUI
/auth/participant	被計測者一覧の返却	GET	WebUI
/auth/participant/<participant_id>	被計測者の更新	POST	WebUI
/auth/participant/<participant_id>	被計測者の返却	GET	WebUI
/auth/google/callback	GoogleのOAuth用	POST	None
/auth/token	長期トークン生成用エンドポイント	POST	WebUI
/auth/token	ユーザが作成したトークン一覧の返却	GET	WebUI
/auth/token/<token_id>	token_idのトークンの更新 (有効無効の切り替え)	POST	WebUI
/auth/token/<token_id>	token_idのトークンの削除	DELETE	WebUI
/auth/jwt/sensors/writejwt	センサデータ書き込み用JWTの発行	POST	トークン認証
/auth/jwt/sensors/readjwt	センサデータ読み込み用のJWTの発行	POST	トークン認証
/auth/jwt/events	イベント管理用JWTの発行	POST	トークン認証
/auth/jwt/service/sensors/readjwt	センサデータ読み込み用のJWTの発行	POST	WebService
/auth/jwt/service/events	イベント管理用JWTの発行	POST	WebService

3.6 バリデーションと規約

- 日付時刻はISO形式の文字列を用いる
- 性別はISO5218に準拠した数値区分を用いる

4 センサ API

4.1 概要

本章は、センサデータの入出力を担うセンサ API の要点 (エンドポイント一覧, データ形式, 圧縮方式, バリデーション) を示す。

ルーティングはNGINXにて/sensor/に振り分けられる。実装内部では/sensorを省くが、本書では実利用を想定し/sensor付きで表記する。

4.2 エンドポイント一覧(概要)

現状の公開エンドポイントはread/writeの2種である。

エンドポイント	説明	メソッド	必要JWT
/sensor/data/write	センサデータの書き込み	POST	sensor write JWT
/sensor/data/read	センサデータの読み出し	POST	sensor read JWT

4.3 データ形式・圧縮

4.3.1 送受信コンテナ(リクエスト/レスポンスの外枠)

formatとcompressionを指定し、本体データはBase64文字列(data)として運ぶ。
書き込み時はリクエストボディ、読み出し時はレスポンスボディに同様の構造で格納される。

```
{  
  "format": "json",  
  "compression": "gzip",  
  "data": "H4sIAAAAAAAAAE8tIzcnJBwCGphA2BQAAAA=="  
}
```

- format: データ形式("json"/"messagepack")
- compression: 圧縮方式("gzip", "lz4", "brotli", "none")
- data: 上記format->compressionを施したバイト列をBase64化した文字列

4.3.2 素データ

dataを展開すると、時系列を共有する可変列のレコード集合になる。

```
{
  "time": [ "", "", ... ],
  "data1": [ 0.1, 0.2, ... ],
  "data2": [ 0.3, null, ... ],
  ....
}
```

- time: タイムスタンプの配列。要素はタイムスタンプ数値, ISO8601文字列のいずれか(統一推奨)。
- それ以外のキー: 任意列。timeと同じ長さの配列で、欠損はnullを許容。
列名は読み出し要求のrows指定に用いる。

4.3.3 formatとcompressionのアルゴリズム

format	変換
json	JSON文字列をUTF-8バイトへ変換
messagepack	MessagePackバイトへ変換

compression	変換
gzip	バイト列をgzip圧縮
lz4	バイト列をLZ4圧縮
brrotli	バイト列をBrotli圧縮
none	圧縮なし

4.4 リクエスト仕様

4.4.1 /sensor/data/write (POST)

要件:

- sensor write JWT.
- リクエストボディにSection 4.3.1の送信コンテナを入れる。

例(JSON + gzip + Base64):

```
{
  "compression": "gzip",
  "data": "H4sIAAAAAAAAAE8tIzcnJBwCGphA2BQAAAA==",
  "format": "json"
}
```

バリデーション(展開後の素データ):

- timeは配列で、要素はタイムスタンプorISO8601文字列(混在非推奨).
- timeと各データ列の長さは一致しなければならない.
- 欠損はnullを許容. NaN等の非JSON値は不可.
- 列名は(キー名)は自由だが、読み出し時のrows名と一致させる必要がある.
- formatとcompressionは上記の列挙値のみを許可.

4.4.2 /sensor/data/read (POST)

要件:

- sensor read JWT.
- リクエストボディで希望する format と compression を指定し、時間範囲と取得列(rows)を与える.

例:

```
{
  "compression": "gzip",
  "end_time": "2025-01-01T01:00:00Z",
  "format": "json",
  "rows": ["EEG_MuseS_TP10", "EEG_MuseS_TP9"],
  "start_time": "2025-01-01T00:00:00Z"
}
```

バリデーション:

- start_time/endtime はISO8601文字列を許可. start_time < endtime.
- rows は非空配列. 要素は書き込み時に用いた列名と一致している必要がある.
- format と compression は列挙値のみを許可.

レスポンス: 要求した format / compression に従い, Section 4.3.1の送受信コンテナで, 圧縮済みBase64として返却される. (クライアント側で解凍->復元が必要).

4.5 実装上の注意

- ルーティング: 本書は/sensor/**を前提に表記する.
- タイムスタンプ: timeの方は数値orISO8601文字列を許容.
- 圧縮・形式: 列挙値(gzip/lz4/brotli/none, json/messagepack)のみ許可
- 互換性: rowsは列名の厳密一致を要する. 表記ゆれに注意.

5 イベント API

5.1 概要

本章は、イベントデータの作成・取得・更新・削除を担うEvent APIの要点(エンドポイント一覧, パラメータ, バリデーション)を示す。ルーティングはNGINXにて/event/に振り分けられる。実装内部では/eventを省略するが、本章では実利用を想定し/event付きで表記する。

5.2 エンドポイント一覧(概要)

エンドポイント	説明	メソッド	必要 JWT 属性
/event/events	イベント作成	POST	event
/event/events	イベント取得(一覧)	GET	event
/event/events/ <event_id>	イベント削除(作成者のみ)	DELETE	event
/event/events/ <event_id>	イベント更新(作成者のみ)	POST	event

5.3 リクエスト仕様・バリデーション

5.3.1 作成: /event/events(POST)

イベントを新規作成する。

要件: event属性のJWT。

Request Body(例):

```
{
  "description": "Morning jogging session",
  "details": {
    "duration": "30min",
    "location": "park"
  },
  "end_time": "2025-01-01T12:30:00Z",
  "event": "Running",
}
```

```

"start_time": "2025-01-01T12:00:00Z",
"user_id": "nanoid"
}

```

フィールド定義:

key	説明
event	イベントを端的に表す文字列
description	追加説明
user_id	対象ユーザID
start_time	開始時刻(ISO8601文字列)
end_time	終了時刻(任意. 未指定時は開始時刻と同一扱い)
details	メタ情報(オブジェクト)

バリデーション

- start_time/end_timeはISO8601文字列を許可. end_time未指定時はstart_timeと同一として扱う.
- start_time <= end_timeを満たすこと.
- detailsは辞書型. 任意キー・任意値を許容.
- user_idは対象ユーザの内部ID. JWTの追加クレームと整合すること.

5.3.2 取得: /event/events (GET)

イベント一覧を返す. クエリで絞り込み可能.

要件: event属性のJWT.

クエリパラメータ:

パラメータ	型	必須	説明
role	experimenter / participant	必須	返却対象の切替 (作成者視点/被計測者視点)
start_time	ISO8601	任意	この時刻以後のイベントを返却
end_time	ISO8601	任意	この時刻以前のイベントを返却

バリデーション:

- roleは表の列挙値のみ許可.
- start_time / end_timeはISO8601. 指定時は start_time <= end_time を満たすこと.

5.3.3 削除: /event/events/<event_id> (DELETE)

要件:

- event属性のJWT.
- イベント作成者のみ削除可能.

バリデーション:

- event_idは既存イベントのIDであること.
- 実行主体が当該イベントの作成者であること.

5.3.4 更新: event/events/<event_id> (POST)

要件:

- event属性のJWT.
- イベント作成者のみ更新可能.

Request Body(例):

```
{
  "description": "Morning jogging session",
  "details": {
    "duration": "30min",
    "location": "park"
  },
  "end_time": "2025-01-01T12:30:00Z",
  "event": "Running",
  "start_time": "2025-01-01T12:00:00Z"
}
```

バリデーション:

- 更新対象のevent_idが有効であること
- 各フィールドの意味は作成時(Section 5.3.1)と同様. start_time/end_timeの整合を保つこと

5.4 実装上の注意

- ルーティング: 本書は/event/**を前提に表記.
- 権限制御: 削除・更新は作成者のみ実行可能. サーバ側で作成者を検証する.
- タイムスタンプ: ISO8601文字列を基本とする. リスponseボディはこの限りではない.

6 WebUI UI仕様

6.1 概要

WebUIはBioDBのユーザ，認証に関わる機能の管理に必要なUIを提供する．主に以下の機能を提供する．

- ・ ユーザの確認・更新
- ・ 被計測者の追加・確認・更新
- ・ 長期トークンの発行・確認・更新

6.2 各画面のUI仕様

ナビゲーションを用いて各機能にアクセスする．

6.2.1 ログイン画面

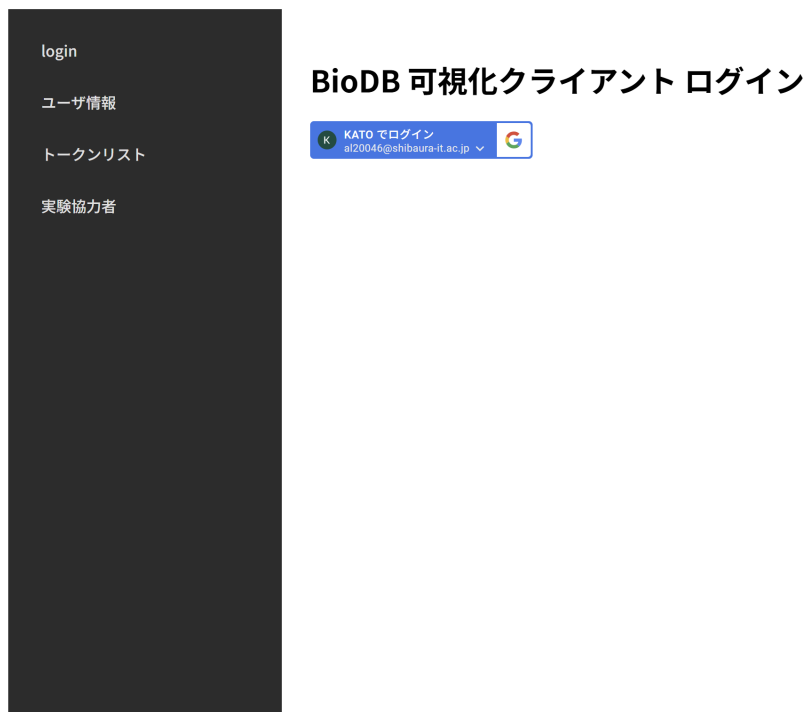


Figure 6.1: webui_login_page

Googleのログインボタンを配置し，ボタンを押すことでログイン用のポップアップが出現する．

ログイン成功の場合は、ユーザ情報ページへ遷移する。

6.2.2 ユーザ情報ページ

The screenshot displays the 'webui_user_info' page. On the left is a dark sidebar menu with the following items: 'login', 'ユーザ情報' (User Information), 'トークンリスト' (Token List), and '実験協力者' (Experiment Collaborator). The main content area has a green header 'ユーザー情報' (User Information). Below this, a dark form contains the following fields: 'メールアドレス:' (Email Address) with the value 'al20046@shibaura-it.ac.jp'; 'ユーザID:' (User ID) with the value 'J9A0iU2p4bsG9DgUSMgAI'; '名前:' (Name) with the value '加藤 颯真'; '性別:' (Gender) with a dropdown menu showing '男性' (Male); and '生年月日:' (Date of Birth) with the value '2001/06/04'. At the bottom of the form is a green button labeled '📄 情報を更新' (Update Information).

Figure 6.2: webui_user_info

ユーザの情報を表示する。メールアドレス、ユーザID、名前、性別、生年月日が表示される。名前、性別、生年月日が更新可能。

6.2.3 被計測者管理画面

被計測者管理画面は2つの画面から成る。
被計測者一覧をリスト表示する画面、被計測者の追加画面の2つである。

被計測者リスト表示



Figure 6.3: webui_participant_list

被計測者のID，メールアドレス，名前，性別，生年月日，有効無効，を表示する。
上のボタンから被計測者追加画面へ遷移する。

管理者のみ各被計測者データの更新が可能である。
青色の編集ボタンを押すことで，名前，性別，生年月日を更新するための編集画面がポップアップされる。
緑色のボタンを押すことで，有効無効を切り替える。

被計測者追加画面

login

ユーザ情報

トークンリスト

実験協力者

< リストに戻る

実験協力者追加画面

Email:

名前:

性別:

未選択

誕生日:

年 / 月 / 日

実験協力者追加

Figure 6.4: webui_participant_create

被計測者の追加を行う画面。
メールアドレス，名前，性別，生年月日を入力した状態で，追加ボタンを押すことで追加される。

6.2.4 長期トークン管理画面

長期トークン管理画面は2つの画面から成る．長期トークン一覧画面，長期トークン作成画面である．

長期トークン一覧画面



Figure 6.5: webui_token_list

長期トークンを一覧表示する。

各長期トークンについてはID，説明，作成日，有効期限，状態，スコープを表示する。
上の新規トークン作成ボタンから長期トークン作成画面へ遷移する。

各トークンについて有効無効の切替と，削除が右のボタンから行える。

長期トークン作成画面



Figure 6.6: webui_token_create

長期トークンの新規追加を行う画面。
トークンの有効期限・追加説明(任意)を入力した状態で、
トークン作成ボタンを押すことで追加される。

7 util UI仕様

7.1 概要

utilではBioDBに格納されているデータの可視化機能を提供する。

7.2 各画面のUI仕様

7.2.1 リアルタイムデータチェッカー

リアルタイムデータチェッカー

被計測者ID:

グラフ追加削除

EEG_MuseS_TP10

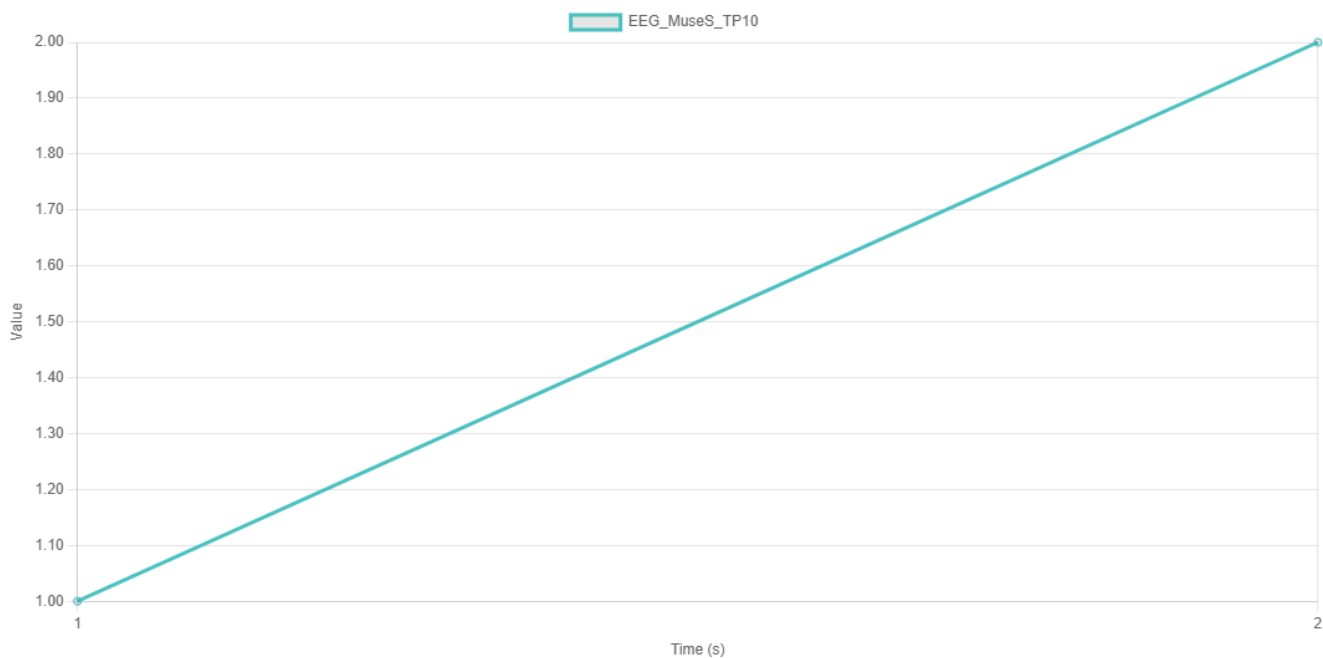


Figure 7.1: util_realtime_data_cheker

被計測者IDの入力ボックス，取得したい時系列名の入力ボックス，時系列の追加・削除ボタン，動的に追加されるグラフで構成される．

被計測者IDが入力されている状態かつ，時系列名を入力した状態で，追加ボタンが押されたときにグラフを追加する．複数グラフの描画も対応する．

グラフがある状態ならば，毎秒BioDBAPIサーバにリクエストを投げ，直近10秒のデータを取得し，描画を行う．

7.2.2 シンプルデータチェッカー

シンプルデータチェッカー

被計測者ID:

タイムゾーン

TimeZones:

グラフ追加削除

時間指定

開始時間: 終了時間:

時系列グラフ

EEG_MuseS_TP10

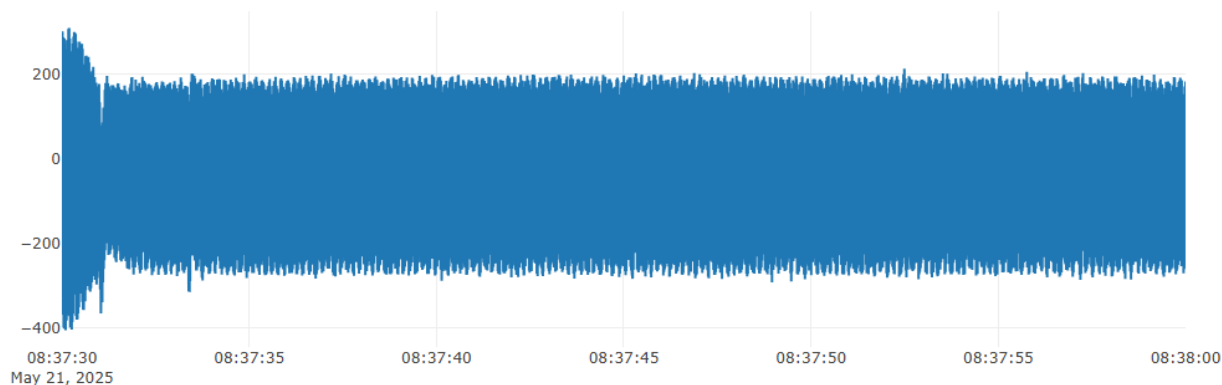


Figure 7.2: util_simple_data_cheker

被計測者IDの入力ボックス，タイムゾーンのセレクトボックス，取得したい時系列名の入力ボックス，時系列の追加・削除ボタン，開始・終了時刻の入力ボックス，データ更新用のPlotボタン，動的に追加されるグラフで構成される．

タイムゾーンはデフォルトでブラウザのタイムゾーンが選択される。

被計測者ID，開始・終了時刻が入力されている状態，かつ確認したい時系列について追加している状態で，plotボタンを押すことでBioDBAPIサーバにリクエストを投げ，データを取得し，描画を行う。

7.2.3 簡易イベント作成

簡易イベント作成ページ

被計測者ID:

タイムゾーン

TimeZones:

イベント情報

イベント名:

開始時刻:

終了時刻:

補足説明:

Figure 7.3: util_event_create

被計測者IDの入力ボックス，タイムゾーンのセレクトボックス，イベント情報の入力ボックス，作成ボタンで構成される。

タイムゾーンはデフォルトでブラウザのタイムゾーンが選択される。

イベント情報の内補足説明以外を埋めた状態でイベント作成ボタンを押すことで，BioDBAPIサーバにリクエストが送られ，イベント情報が保管される。

7.2.4 簡易イベント編集ページ

簡易イベント編集ページ

被計測者ID:

タイムゾーン

TimeZones:

時刻指定

開始時間: 終了時間:

イベント	開始時刻	終了時刻	補足
TaskA_Start	2025-06-27 16:05:00	2025-06-27 16:05:00	Marker for test scenario.
TaskA_End	2025-06-27 16:10:00	2025-06-27 16:10:00	Marker for test scenario.
TaskB_Start	2025-06-27 16:15:00	2025-06-27 16:15:00	Marker for test scenario.
TaskB_End	2025-06-27 16:20:00	2025-06-27 16:20:00	Marker for test scenario.

イベント選択:

イベント情報

イベント名:
開始時刻:
終了時刻:
補足説明:

Figure 7.4: util_event_edit

被計測者IDの入力ボックス，タイムゾーンのセレクトボックス，開始・終了時刻の入力ボックス，イベント取得ボタン，イベントリストの表示テーブル，イベント選択用セレクトボックス，イベント情報入力ボックス，イベント更新・削除ボタンで構成される。

タイムゾーンはデフォルトでブラウザのタイムゾーンが選択される。

被計測者ID，開始・終了時刻を入力した状態で，イベント取得ボタンを押すことでイベントのテーブルが表示される。
任意のイベントを選択することで，下のイベント情報入力ボックスにイベント情報が入力される。
その状態でイベントの更新・削除を行える。

7.2.5 イベントデータと時系列プロット

イベントデータと時系列プロット

被計測者ID:

タイムゾーン

TimeZones:

時刻指定

開始時間: 終了時間:

イベント	開始時刻	終了時刻	補足
rest	2025-05-21T17:37:28+09:00	2025-05-21T17:38:45+09:00	
music	2025-05-21T17:39:35+09:00	2025-05-21T17:42:03+09:00	ZegalltA
music	2025-05-21T17:42:33+09:00	2025-05-21T17:44:27+09:00	Keep it Up
music	2025-05-21T17:45:18+09:00	2025-05-21T17:49:17+09:00	Lost World
rest	2025-05-26T09:27:34+09:00	2025-05-26T09:29:05+09:00	

イベント選択:

グラフ追加削除

時系列グラフ

EEG_MuseS_TP10

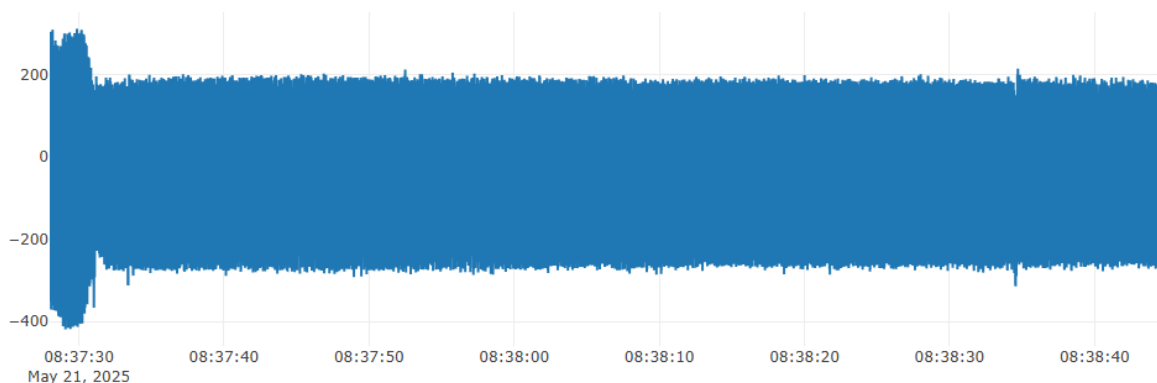


Figure 7.5: util_event_plot

被計測者IDの入力ボックス, タイムゾーンのセレクトボックス, 開始・終了時刻の入力ボックス, イベント取得ボタン, イベントリストの表示テーブル, イベント選択用セレクトボックス,

取得したい時系列名の入力ボックス，時系列の追加・削除ボタン，動的に追加されるグラフで構成される．
タイムゾーンはデフォルトでブラウザのタイムゾーンが選択される．
被計測者ID，開始・終了時刻を入力した状態で，
イベント取得ボタンを押すことでイベントのテーブルが表示される．
任意のイベントを選択した状態で，時系列を追加したタイミングでデータを取得し，描画を行う．

7.2.6 イベントデータと感情マップ

イベントデータと感情マップ

被計測者ID:

使用機材

脳波計: 脈波計:

タイムゾーン

TimeZones:

時刻指定

開始時間: 終了時間:

イベント	開始時刻	終了時刻	補足
rest	2025-05-21T17:37:28+09:00	2025-05-21T17:38:45+09:00	
music	2025-05-21T17:39:35+09:00	2025-05-21T17:42:03+09:00	ZegalltA
music	2025-05-21T17:42:33+09:00	2025-05-21T17:44:27+09:00	Keep it Up
music	2025-05-21T17:45:18+09:00	2025-05-21T17:49:17+09:00	Lost World
rest	2025-05-26T09:27:34+09:00	2025-05-26T09:29:05+09:00	

プロットイベント選択

可視化	イベント	開始時刻	終了時刻	補足
☐	rest	2025-05-21T17:37:28+09:00	2025-05-21T17:38:45+09:00	
☒	music	2025-05-21T17:39:35+09:00	2025-05-21T17:42:03+09:00	ZegalltA
☒	music	2025-05-21T17:42:33+09:00	2025-05-21T17:44:27+09:00	Keep it Up
☒	music	2025-05-21T17:45:18+09:00	2025-05-21T17:49:17+09:00	Lost World
☒	rest	2025-05-26T09:27:34+09:00	2025-05-26T09:29:05+09:00	

安静イベント選択:

感情マップ

脳波指標: / 心拍指標:

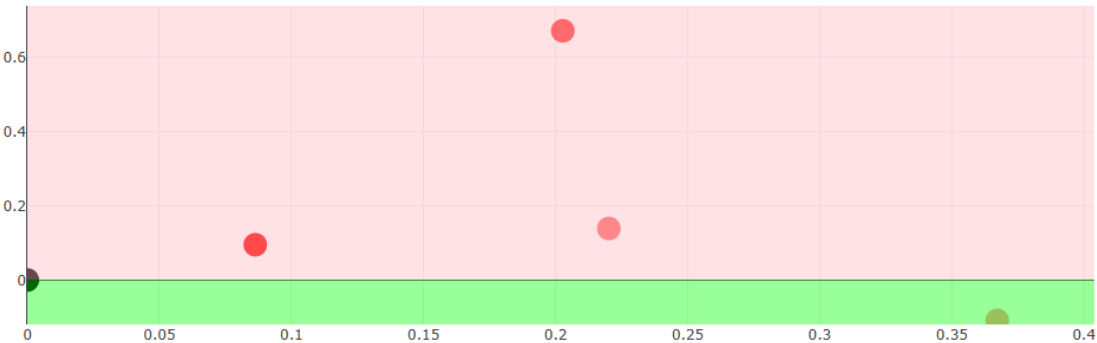


Figure 7.6: util_event_emotionmap

被計測者IDの入力ボックス，使用した脳波計，脈波計のセレクトボックス，タイムゾーンのセレクトボックス，開始・終了時刻の入力ボックス，イベント取得ボタン，イベントリストの表示テーブル，感情マップで描画するイベントの選択用テーブル，安静イベントの選択ボックス，使用する脳波指標，心拍変動指標の選択ボックス，生成される感情マップで構成される．

被計測者ID，使用機材，開始・終了時刻を入力した状態で，イベント取得ボタンを押すことでイベントのテーブルが表示される．プロットイベントの選択と安静イベントの選択，使用する指標が選択されると感情マップの描画を行う．

8 付録(OpenAPI仕様)

8.1 認証API

BioDB Auth API Server (0.0.1)

Download OpenAPI specification: [Download](#)

API documentation for the BioDB Auth API Server

User

ユーザー情報取得 API

認証済みのユーザーが、自分の情報（ID，メールアドレス，名前，性別，生年月日）を取得するためのAPI.

AUTHORIZATIONS: > API Key: *BearerAuth*

Responses

✓ **200** ユーザー情報の取得成功

RESPONSE SCHEMA: */*

id	string [A-Za-z0-9_-]{21} ユーザーID
email	string ユーザーのメールアドレス
name	string ユーザーの名前
sex	integer Enum: 0 1 2 9

	0: 不明, 1: 男性, 2: 女性, 9: 適用不能
└ birthdate	string <date> ユーザーの生年月日

▼ **400** 不正なリクエスト (JWTエラーまたはユーザーが見つからない場合)

RESPONSE SCHEMA: */*

└ code	integer
└ message	string

GET /user/info



ユーザー情報更新 API

認証済みのユーザーが、自分の情報（名前、性別、生年月日）を更新するためのAPI.

AUTHORIZATIONS: > API Key: *BearerAuth*

REQUEST BODY SCHEMA: application/json
required

└ name	string ユーザーの名前
└ sex	integer Enum: 0 1 2 9 0: 不明, 1: 男性, 2: 女性, 9: 適用不能
└ birthdate	string <date> ユーザーの生年月日

Responses

▼ 200 ユーザー情報の更新成功

RESPONSE SCHEMA: */*	
code	integer
message	string

▼ 400 不正なリクエスト

RESPONSE SCHEMA: */*	
code	integer
message	string

▼ 500 サーバー内部エラー（データ更新失敗）

RESPONSE SCHEMA: */*	
code	integer
message	string

POST

/user/info

▼

Request samples

Payload

Content type

application/json

Copy

```
{
  "name": "John Doe",
  "sex": 1,
  "birthdate": "1990-01-01"
}
```

ユーザー作成 API

認証済みの管理者ユーザーが、新しいユーザーを作成するためのAPI。作成されたユーザーはデフォルトで "normal" ロールを持つ。

AUTHORIZATIONS: > API Key: *BearerAuth*

REQUEST BODY SCHEMA: application/json
required

email required	string <email>
role required	string Value: "normal" ユーザーのロール（現在は 'normal' のみサポート）

Responses

✓ **200** ユーザー作成成功

RESPONSE SCHEMA: */*

code	integer
message	string

✓ **400** 不正なリクエスト (JWTエラー、JSONスキーマエラー、または権限エラー)

RESPONSE SCHEMA: */*

code	integer
message	string

✓ **500** サーバー内部エラー（ユーザー作成失敗）

RESPONSE SCHEMA: */*

code	integer
message	string

POST /user



Request samples

Payload

Content type
application/json

Copy

```
{
  "email": "user@example.com",
  "role": "normal"
}
```

Participant

被計測者作成 API

認証済みのユーザーが、新しい被計測者を作成するためのAPI.

AUTHORIZATIONS: > API Key: *BearerAuth*

REQUEST BODY SCHEMA: application/json

required

email required	string <email>
name	string [2 .. 36] characters 被計測者の名前（オプション）
sex	integer Enum: 0 1 2 9 被計測者の性別（オプション） 0: 不明, 1: 男性, 2: 女性, 9: 適用不能
birthdate	string <date> 被計測者の生年月日（オプション）

Responses

✓ **200** 被計測者作成成功

RESPONSE SCHEMA: */*

code	integer
message	string

✓ **400** 不正なリクエスト（JWTエラー、JSONスキーマエラー）

RESPONSE SCHEMA: */*

code	integer
message	string

✓ **500** サーバー内部エラー（被計測者作成失敗）

RESPONSE SCHEMA: */*

code	integer
message	string

POST /participant



Request samples

Payload

Content type

application/json

Copy

```
{
  "email": "participant@example.com",
  "name": "Participant Name",
  "sex": 1,
  "birthdate": "2000-01-01"
}
```

被計測者一覧取得 API

認証済みのユーザーが、すべての被計測者の一覧を取得するためのAPI。

AUTHORIZATIONS: > API Key: *BearerAuth*

Responses

✓ **200** 被計測者一覧の取得成功

RESPONSE SCHEMA: ***/***

└─ code	integer
└─ message	string
└─ participants ▾	Array of objects

Array [

└─ id	string [A-Za-z0-9_-]{21}
└─ email	string <email>
└─ name	string 被計測者の名前
└─ sex	integer Enum: 0 1 2 9 被計測者の性別 0: 不明, 1: 男性, 2: 女性, 9: 適用不能
└─ birth_date	string <date> 被計測者の生年月日
└─ is_enable	boolean 被計測者の有効/無効

]

▼ **400** 認証エラー

RESPONSE SCHEMA: ***/***

└─ code	integer
└─ message	string

▼ **500** サーバーエラー

RESPONSE SCHEMA: ***/***

code	integer
message	string

GET /participant



被計測者情報更新 API

認証済みのユーザーが、指定された被計測者の情報を更新するためのAPI.

AUTHORIZATIONS: > API Key: *BearerAuth*

PATH PARAMETERS

→ participant_id string [A-Za-z0-9_-]{21}
required 更新する対象の被計測者ID

REQUEST BODY SCHEMA: application/json
required

name	string
sex	integer Enum: 0 1 2 9 被計測者の性別 0: 不明, 1: 男性, 2: 女性, 9: 適用不能
birth_date	string <date> 被計測者の生年月日
is_enable	boolean 被計測者の有効/無効

Responses

✓ **200** 被計測者情報の更新成功

RESPONSE SCHEMA: */*

code	integer
message	string

✓ **400** 不正なリクエスト (JWTエラーまたはJSONスキーマエラー)

RESPONSE SCHEMA: */*

code	integer
message	string

✓ **500** サーバー内部エラー (データ更新失敗)

RESPONSE SCHEMA: */*

code	integer
message	string

POST /participant/{participant_id}



Request samples

Payload

Content type
application/json

Copy

```
{
  "name": "Participant Name",
  "sex": 1,
```

```
"birth_date": "2000-01-01",  
"is_enable": true  
}
```

被計測者情報取得 API

認証済みのユーザーが、指定された被計測者の情報を取得するためのAPI.

AUTHORIZATIONS: > API Key: *BearerAuth*

PATH PARAMETERS

→ participant_id string [A-Za-z0-9_-]{21}
 required 取得する対象の被計測者ID

Responses

✓ **200** 被計測者情報の取得成功

RESPONSE SCHEMA: */*

└─ code integer
└─ message string
└─ participant ▾ object

└─ id string [A-Za-z0-9_-]{21}
└─ email string <email>

GET /participant/{participant_id} ▼

Authentication

Google OAuth コールバック API

Google OAuth の認証コードを受け取り、検証後にユーザー情報を取得し、JWT アクセストークンを発行する。ユーザーが未登録の場合はエラー

AUTHORIZATIONS: > API Key: *GoogleIdToken*

HEADER PARAMETERS

→ Authorization **required** string
Google OAuth 認証コード（Authorization ヘッダーの Bearer トークンとして送信）

REQUEST BODY SCHEMA: application/json
required

→ role **required** string
Enum: "manage" "service"
要求するJWTの種類.これによってJWTの権限範囲が変わる.

Responses

✓ **200** 認証成功（JWT アクセストークンを発行）

RESPONSE SCHEMA: */*

→ access_token string

▼ 400

不正なリクエスト（email 情報が取得できなかった場合, roleを指定していない場合, roleが想定外の入力の場合）

RESPONSE SCHEMA: */*

└─ code integer

└─ message string

▼ 401 認証エラー（Google の OAuth 検証に失敗した場合）

RESPONSE SCHEMA: */*

└─ code integer

└─ message string

POST /google/callback



Request samples

Payload

Content type

application/json

Copy

```
{
  "role": "manage"
}
```

長期トークン発行 API

認証済みのユーザーが、新しい長期トークンを発行するための API。トークンは指定したスコープ (`scopes`) で利用可能。 `expiration_days` の範囲内でトークンの有効期限を設定できる。

AUTHORIZATIONS: > API Key: *BearerAuth*

REQUEST BODY SCHEMA: application/json
`required`

scopes <code>required</code>	Array of strings このトークンで許可されるスコープの一覧
expiration_days <code>required</code>	integer [1 .. 365] トークンの有効期間（日数）
description	string トークンの補足情報（オプション）

Responses

✓ **200** トークン発行成功

RESPONSE SCHEMA: `/**`

token	string [A-Za-z0-9_-]{43,44}
-------	-----------------------------

✓ **400** 不正なリクエスト (JSON スキーマエラー、JWTエラー)

RESPONSE SCHEMA: `/**`

code	integer
message	string

✓ **500** サーバー内部エラー（トークン生成失敗）

RESPONSE SCHEMA: */*

code	integer
message	string

POST /token



Request samples

Payload

Content type

application/json

Copy Expand all Collapse all

```
{
  - "scopes": [
    "all"
  ],
  "expiration_days": 30,
  "description": "This is an API token for data access"
}
```

長期トークン一覧取得 API

認証済みのユーザーが、自身の発行した長期トークンの一覧を取得するAPI。JWTによる認証が必要であり、WebUIのJWTである必要がある。

AUTHORIZATIONS: > API Key: *BearerAuth*

Responses

✓ 200 トークン一覧の取得成功

RESPONSE SCHEMA: */*

→ tokens ▾ Array of objects

Array [

token_id	string [0-9a-f]{64} トークンの一意な識別子
user_id	string [A-Za-z0-9_-]{21} このトークンを所有するユーザーのID
created_at	string トークンの作成日時 (MongoDBのフォーマット)
expired_at	string トークンの有効期限 (MongoDBのフォーマット)
scopes	Array of strings このトークンで許可されるスコープの一覧
is_active	boolean トークンの有効状態 (True: 有効, False: 無効)
description	string トークンの補足情報 (オプション)

]

✓ 400 不正なリクエスト (JWT エラー)

RESPONSE SCHEMA: */*

code	integer
message	string

GET /token



長期トークン削除 API

認証済みのユーザーが、自身の発行した長期トークンを削除するための API。JWT による認証が必要であり、対象の `tokenid` が現在のユーザーのものである必要がある。

AUTHORIZATIONS: > API Key: *BearerAuth*

PATH PARAMETERS

→ <code>tokenid</code> <code>required</code>	string <code>[0-9a-f]{64}</code> 削除する対象のトークンID
---	---

Responses

✓ **200** トークン削除成功

RESPONSE SCHEMA: `*/*`

code	integer
message	string

✓ **400** 不正なリクエスト (JWT エラー または 削除失敗)

RESPONSE SCHEMA: `*/*`

code	integer
message	string

DELETE /token/{tokenid}



長期トークン更新 API

認証済みのユーザーが、自身の発行した長期トークンの状態 (`is_active`) を更新するための API. JWT による認証が必要であり、対象の `tokenid` が現在のユーザーのものである必要がある.

AUTHORIZATIONS: > API Key: *BearerAuth*

PATH PARAMETERS

→ `tokenid`
`required` string `[0-9a-f]{64}`
更新する対象のトークンID

REQUEST BODY SCHEMA: `application/json`
`required`

→ `is_active`
`required` boolean
トークンの有効状態 (True: 有効, False: 無効)

Responses

✓ **200** トークン更新成功

RESPONSE SCHEMA: `*/*`

┌ `code` integer
└ `message` string

✓ **400** 不正なリクエスト (JWT エラー、JSON スキーマエラー、または更新失敗)

RESPONSE SCHEMA: `*/*`

code	integer
message	string

POST /token/{tokenid}

Request samples

Payload

Content type
application/json

Copy

```
{
  "is_active": true
}
```

センサーデータ書き込み用 JWT 発行 API

認証済みのユーザーが、センサーデータの書き込み権限を持つJWTを発行するAPI。提供された長期トークン (`token`) のスコープを検証し、適切な権限がある場合にJWTを発行する。 `start_time` から `end_time` の範囲でのアクセスが可能になる。

REQUEST BODY SCHEMA: application/json
required

user_id required	string [A-Za-z0-9_-]{21} リクエストを行うユーザーのID
token required	string [A-Za-z0-9_-]{43,44} ユーザーの認証用長期トークン
participant_id required	string [A-Za-z0-9_-]{21} センサーデータを書き込む対象の参加者ID

start_time required	string <date-time> JWT の有効開始時間
end_time required	string <date-time> JWT の有効終了時間

Responses

✓ 200 JWT 発行成功

RESPONSE SCHEMA: */*

code	integer
message	string
jwt	string 発行された JWT

✓ 400 不正なリクエスト (JSON スキーマエラー または 無効なトークン)

RESPONSE SCHEMA: */*

code	integer
message	string

✓ 401 認証エラー (権限なし)

RESPONSE SCHEMA: */*

code	integer
message	string

POST /jwt/sensors/writejwt



Request samples

Payload

Content type

application/json

Copy

```
{
  "user_id": "AAAAAAAAAAAAAAAAAAAA",
  "token": "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA",
  "participant_id": "AAAAAAAAAAAAAAAAAAAA",
  "start_time": "2025-01-01T00:00:00Z",
  "end_time": "2025-01-01T01:00:00Z"
}
```

センサーデータ読み取り用 JWT 発行 API

認証済みのユーザーが、センサーデータの読み取り権限を持つ JWT を発行する API。提供された長期トークン (`token`) のスコープを検証し、適切な権限がある場合に JWT を発行する。 `start_time` から `end_time` の範囲でのアクセスが可能になる。

REQUEST BODY SCHEMA: application/json

required

<code>user_id</code> required	string [A-Za-z0-9_-]{21} リクエストを行うユーザーのID
<code>token</code> required	string [A-Za-z0-9_-]{43,44} ユーザーの認証用長期トークン
<code>participant_id</code> required	string [A-Za-z0-9_-]{21} センサーデータを読み取る対象の参加者ID

start_time required	string <date-time> JWT の有効開始時間
end_time required	string <date-time> JWT の有効終了時間

Responses

✓ 200 JWT 発行成功

RESPONSE SCHEMA: */*

jwt	string 発行された JWT
-----------------------------	--

✓ 400 不正なリクエスト (JSON スキーマエラー または 無効なトークン)

RESPONSE SCHEMA: */*

code	integer
message	string

✓ 401 認証エラー (権限なし)

RESPONSE SCHEMA: */*

code	integer
message	string

POST /jwt/sensors/readjwt



Request samples

Payload

Content type

application/json

Copy

```
{
  "user_id": "AAAAAAAAAAAAAAAAAAAA",
  "token": "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA",
  "participant_id": "AAAAAAAAAAAAAAAAAAAA",
  "start_time": "2025-01-01T00:00:00Z",
  "end_time": "2025-01-01T01:00:00Z"
}
```

イベントデータ用 JWT 発行 API

認証済みのユーザーが、イベントデータの取得権限を持つ JWT を発行する API。提供された長期トークン (`token`) のスコープを検証し、適切な権限がある場合に JWT を発行する。 `start_time` から `end_time` の範囲でのアクセスが可能になる。

REQUEST BODY SCHEMA: application/json

required

user_id required	string [A-Za-z0-9_-]{21} リクエストを行うユーザーのID
token required	string [A-Za-z0-9_-]{43,44} ユーザーの認証用長期トークン
participant_id required	string [A-Za-z0-9_-]{21} イベントデータを取得する対象の参加者ID
start_time required	string <date-time> JWT の有効開始時間
end_time required	string <date-time> JWT の有効終了時間

Responses

▼ 200 JWT 発行成功

RESPONSE SCHEMA: */*	
→ jwt	string 発行された JWT

▼ 400 不正なリクエスト (JSON スキーマエラー または 無効なトークン)

RESPONSE SCHEMA: */*	
└ code	integer
└ message	string

▼ 401 認証エラー (権限なし)

RESPONSE SCHEMA: */*	
└ code	integer
└ message	string

POST /jwt/events

▼

Request samples

Payload

Content type

application/json

Copy

```
{  
  "user_id": "AAAAAAAAAAAAAAAAAAAA",  
}
```



```
{
  "token": "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA",
  "participant_id": "AAAAAAAAAAAAAAAAAAAAAAAA",
  "start_time": "2025-01-01T00:00:00Z",
  "end_time": "2025-01-01T01:00:00Z"
}
```

センサーデータ読み取り用 JWT 発行 API

認証済みのユーザーが、センサーデータの読み取り権限を持つ JWT を発行する API。WebService スコープを持つ JWT で利用可能。 `start_time` から `end_time` の範囲でのアクセスが可能になる。

AUTHORIZATIONS: > API Key: *BearerAuth*

REQUEST BODY SCHEMA: application/json
required

→ user_id required	string [A-Za-z0-9_-]{21} リクエストを行うユーザーのID
→ participant_id required	string [A-Za-z0-9_-]{21} センサーデータを読み取る対象の参加者ID
→ start_time required	string <date-time> JWT の有効開始時間
→ end_time required	string <date-time> JWT の有効終了時間

Responses

✓ 200 JWT 発行成功

RESPONSE SCHEMA: */*

→ jwt	string 発行された JWT
-------	---------------------

▼ **400** 不正なリクエスト (JSON スキーマエラー または 無効なトークン)

RESPONSE SCHEMA: */*

code	integer
message	string

▼ **401** 認証エラー (権限なし)

RESPONSE SCHEMA: */*

code	integer
message	string

POST /jwt/service/sensors/readjwt

Request samples

Payload

Content type

application/json

Copy

```
{
  "user_id": "AAAAAAAAAAAAAAAAAAAA",
  "participant_id": "AAAAAAAAAAAAAAAAAAAA",
  "start_time": "2025-01-01T00:00:00Z",
  "end_time": "2025-01-01T01:00:00Z"
}
```

認証済みのユーザーが、イベントデータの権限を持つ JWT を発行する API。WebService スコープを持つ JWT で利用可能。 `start_time` から `end_time` の範囲でのアクセスが可能になる。

AUTHORIZATIONS: > API Key: *BearerAuth*

REQUEST BODY SCHEMA: application/json
required

→ user_id required	string [A-Za-z0-9_-]{21} リクエストを行うユーザーのID
→ participant_id required	string [A-Za-z0-9_-]{21} センサーデータを読み取る対象の参加者ID
→ start_time required	string <date-time> JWT の有効開始時間
→ end_time required	string <date-time> JWT の有効終了時間

Responses

✓ 200 JWT 発行成功

RESPONSE SCHEMA: */*

→ jwt	string 発行された JWT
-------	---------------------

✓ 400 不正なリクエスト (JSON スキーマエラー または 無効なトークン)

RESPONSE SCHEMA: */*

→ code	integer
→ message	string

✓ 401 認証エラー (権限なし)

RESPONSE SCHEMA: */*

code	integer
message	string

POST /jwt/service/events



Request samples

Payload

Content type

application/json

Copy

```
{
  "user_id": "AAAAAAAAAAAAAAAAAAAA",
  "participant_id": "AAAAAAAAAAAAAAAAAAAA",
  "start_time": "2025-01-01T00:00:00Z",
  "end_time": "2025-01-01T01:00:00Z"
}
```

8.2 センサAPI

BioDB Victoria Sensor API Server (0.1.0)

Download OpenAPI specification: [Download](#)

API documentation for the BioDB Victoria Sensor API Server

センサデータの読み出し

Victoria Metricsからセンサデータを読み出します。JWTのroleはsensor_readである必要があります。

AUTHORIZATIONS: > HTTP: *HTTPBearer*

REQUEST BODY SCHEMA: application/json

required

compression required	string (Compression) Enum: "gzip" "lz4" "brotli" "none"
format required	string (Format) Enum: "json" "messagepack"
rows required	Array of strings (Rows)
start_time required	string <date-time> (Start Time)
end_time required	string <date-time> (End Time)

Responses

✓ **200** Successful Response

RESPONSE SCHEMA: application/json

compression string (Compression)

required	Enum: "gzip" "lz4" "brotli" "none"
format required	string (Format) Enum: "json" "messagepack"
data required	string (Data) Base64 encoded compressed data

▼ **422** Validation Error

RESPONSE SCHEMA: application/json

→ detail ▼ Array of objects (Detail)

Array [	
loc ▼ required	Array of strings or integers (Location)
Array [Any of string integer string]	
msg required	string (Message)
type required	string (Error Type)
]	

POST /data/read ▼

Request samples

Payload

Content type

application/json

Copy Expand all Collapse all

```
{
  "compression": "gzip",
  "format": "json",
  - "rows": [
    "string"
  ],
  "start_time": "2019-08-24T14:15:22Z",
  "end_time": "2019-08-24T14:15:22Z"
}
```

Response samples**200****422****Content type**

application/json

Copy

```
{
  "compression": "gzip",
  "format": "json",
  "data": "string"
}
```

センサーデータの書き込み

Victoria Metricsにセンサーデータを書き込みます。JWTのroleはsensor_writeである必要があります。

AUTHORIZATIONS: > HTTP: *HTTPBearer*

REQUEST BODY SCHEMA: application/json
required

compression required	string (Compression) Enum: "gzip" "lz4" "brotli" "none"
format required	string (Format) Enum: "json" "messagepack"
data required	string (Data) Base64 encoded compressed data

Responses

✓ 200 Successful Response

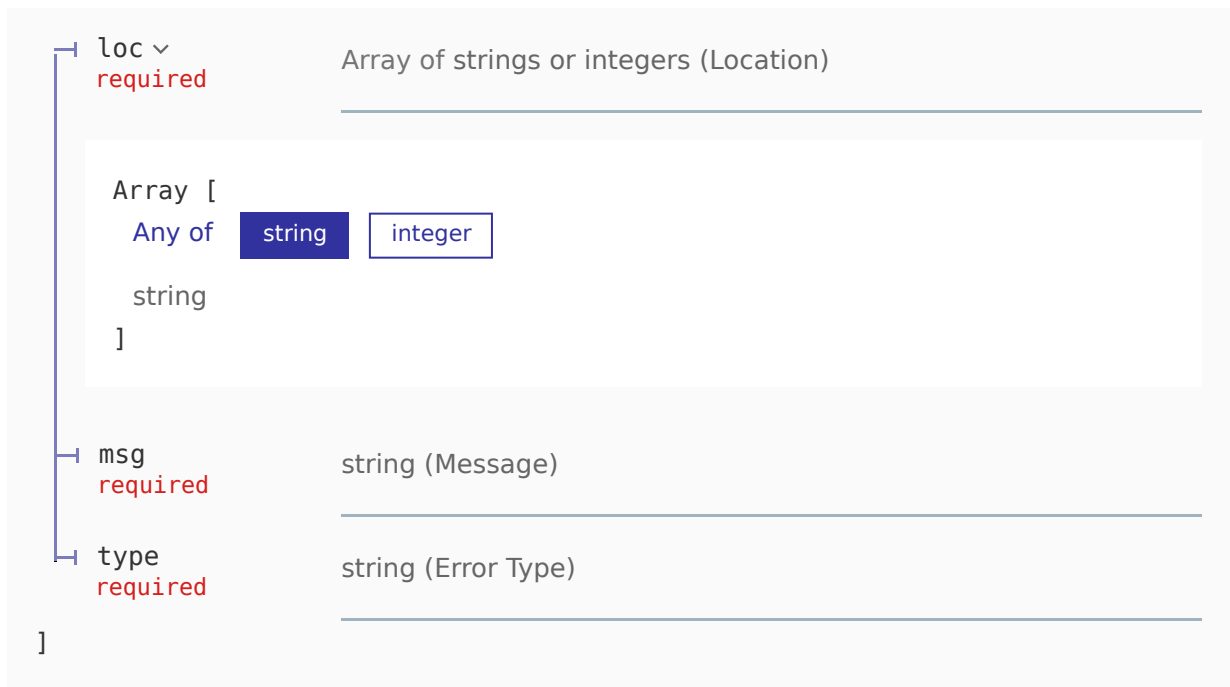
RESPONSE SCHEMA: application/json

status required	string (Status) Enum: "success" "error"
message ▾	Message (string) or Message (null) (Message)
Any of Message Message string (Message)	
code ▾	Code (integer) or Code (null) (Code)
Any of Code Code integer (Code)	

✓ 422 Validation Error

RESPONSE SCHEMA: application/json

detail ▾	Array of objects (Detail)
Array [	



POST /data/write

Request samples

Payload

Content type
application/json

Copy

```
{
  "compression": "gzip",
  "format": "json",
  "data": "string"
}
```

Response samples

200

422

Content type
application/json

Copy

```
{  
  "status": "success",  
  "message": "string",  
  "code": 0  
}
```

8.3 イベントAPI

BioDB Event API Server (0.0.1)

Download OpenAPI specification: [Download](#)

API documentation for the BioDB Event API Server

Events

イベント作成 API

認証済みのユーザーが、新しいイベントを作成する API。 イベントを作成するには `event` 権限を持つ JWT が必要であり、JWT に指定された `start_time` から `end_time` の範囲内である必要がある。

AUTHORIZATIONS: > API Key: *BearerAuth*

REQUEST BODY SCHEMA: application/json
required

user_id required	string [A-Za-z0-9_-]{21} イベントの対象ユーザー ID
start_time required	string <date-time> イベントの開始時間
end_time	string <date-time> イベントの終了時間（オプション）
event required	string イベントの種類（例: 'Running', 'Sleeping', 'Eating'）
description	string イベントの説（オプション）
details	object

Responses

✓ 200 イベント作成成功

RESPONSE SCHEMA: */*

code	integer
message	string
event_id	string <uuid> 作成されたイベントの ID

✓ 400 不正なリクエスト（JWT エラー、リクエストボディエラー）

RESPONSE SCHEMA: */*

code	integer
message	string

✓ 500 サーバー内部エラー（イベント作成失敗）

RESPONSE SCHEMA: */*

error	string
-------	--------

POST /events



Request samples

Payload

Content type

application/json

Copy Expand all Collapse all

```
{
  "user_id": "AAAAAAAAAAAAAAAAAAAA",
  "start_time": "2025-01-01T12:00:00Z",
  "end_time": "2025-01-01T12:30:00Z",
  "event": "Running",
  "description": "Morning jogging session",
  - "details": {
    "location": "park",
    "duration": "30min"
  }
}
```

イベント取得 API

認証済みのユーザーが、指定された条件に一致するイベントの一覧を取得する API。取得には **event** 権限を持つ JWT が必要であり、JWT に指定された **start_time** から **end_time** の範囲内である必要がある。

AUTHORIZATIONS: > API Key: *BearerAuth*

QUERY PARAMETERS

role	string
required	Enum: "experimenter" "participant"
	取得するイベントの対象 (experimenter : 実行者が作成者のイベント, participant : 対象ユーザーのイベント)
start_time	string <date-time> 取得開始時間（指定がない場合は JWT の start_time）
end_time	string <date-time> 取得終了時間（指定がない場合は JWT の end_time）

Responses

✓ 200 イベント取得成功

RESPONSE SCHEMA: ***/***

code	integer
message	string
event_list ✓	Array of objects

Array [	
event_id	string <uuid> イベント ID
user_id	string [A-Za-z0-9_-]{21} イベントの対象ユーザー ID
start_time	string <date-time>
end_time	string <date-time>
event	string イベントの種類
description	string イベントの説明
created_by	string [A-Za-z0-9_-]{21} イベント作成者の ID
]	

✓ 400 不正なリクエスト (JWT エラー、クエリパラメータエラー)

RESPONSE SCHEMA: ***/***

code	integer
message	string

✓ 500 サーバー内部エラー (イベント取得失敗)

RESPONSE SCHEMA: ***/***

code	integer
message	string

GET /events



イベント削除 API

認証済みのユーザーが、自分が作成したイベントを削除する API。削除には **event** 権限を持つ JWT が必要であり、JWT に指定された **start_time** から **end_time** の範囲内である必要がある。削除対象のイベントの作成者 (**created_by**) が現在のユーザーと一致している場合のみ削除可能。

AUTHORIZATIONS: > API Key: *BearerAuth*

PATH PARAMETERS

event_id required	string <uuid> 削除対象のイベント ID
-----------------------------	-------------------------------

Responses

✓ **200** イベント削除成功

RESPONSE SCHEMA: ***/***

code	integer
message	string

▼ 400 不正なリクエスト (JWT エラー)

RESPONSE SCHEMA: */*

code	integer
message	string

▼ 401 権限エラー (イベントが存在しない, 削除権限なし)

RESPONSE SCHEMA: */*

code	integer
message	string

▼ 500 サーバー内部エラー (イベント削除失敗)

RESPONSE SCHEMA: */*

code	integer
message	string

DELETE /events/{event_id}



イベント更新 API

認証済みのユーザーが、指定されたイベントの情報を更新する API。更新には `event` 権限を持つ JWT が必要であり、JWT に指定された `start_time` から `end_time` の範囲内である必要がある。また、イベントの作成者 (`created_by`) が現在のユーザーと一致している場合のみ更新が可能。更新可能なフィールドは `start_time` , `end_time` , `event` , `description` , `details` である。

AUTHORIZATIONS: > API Key: *BearerAuth*

PATH PARAMETERS

→ event_id required	string 更新対象のイベント ID
------------------------	------------------------

REQUEST BODY SCHEMA: application/json
required

→ start_time	string <date-time> 更新後のイベント開始時間（オプション）
→ end_time	string <date-time> 更新後のイベント終了時間（オプション）
→ event	string 更新後のイベントの種類（オプション）
→ description	string 更新後のイベントの説明（オプション）
→ details	object 更新後のイベント詳細情報（オプション）

Responses

✓ 200 イベント更新成功

RESPONSE SCHEMA: */*

→ code	integer
→ message	string

✓ 400 不正なリクエスト（JWT エラー、リクエストボディエラー、無効な更新）

RESPONSE SCHEMA: */*

→ code	integer
--------	---------

└─ message string

▼ **401** 権限エラー（更新権限なし）

RESPONSE SCHEMA: */*

└─ code integer

└─ message string

▼ **500** サーバー内部エラー（イベント更新失敗）

RESPONSE SCHEMA: */*

└─ code integer

└─ message string

POST /events/{event_id}



Request samples

Payload

Content type

application/json

Copy Expand all Collapse all

```
{
  "start_time": "2025-01-01T12:00:00Z",
  "end_time": "2025-01-01T12:30:00Z",
  "event": "Running",
  "description": "Morning jogging session",
  "details": {
    "location": "park",
    "duration": "30min"
  }
}
```

}